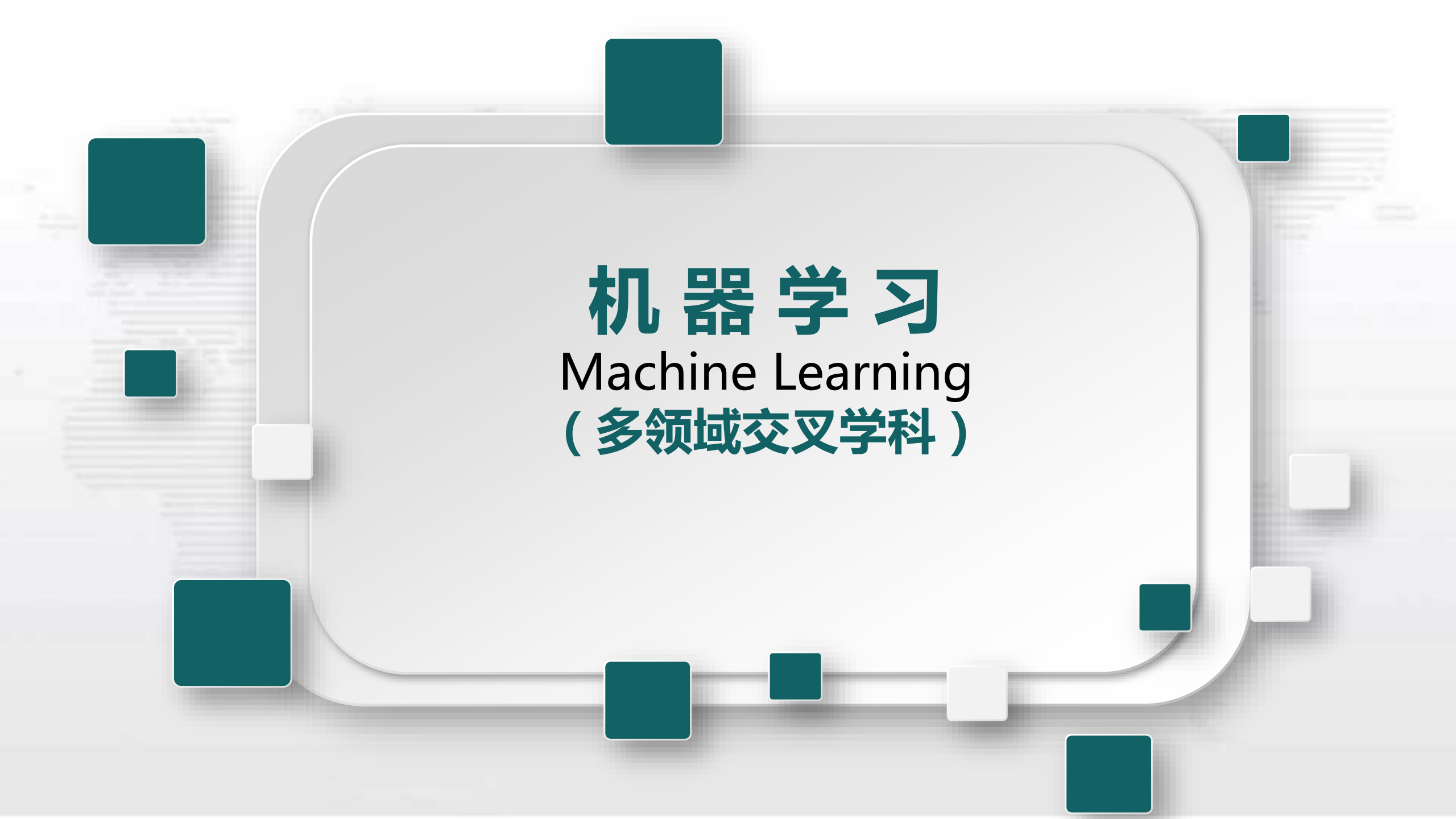




机器学习

Machine Learning
(多领域交叉学科)



目录

CONTENTS

1

机器学习简介

2

机器学习相关基础知识

3

K近邻算法 (KNN)

4

线性回归法

5

主成分分析法 (PCA)

6

逻辑回归法

7

支持向量机算法 (SVM)

8

决策树算法

9

集成学习和随机森林

1

线性回归算法

- 解决回归问题
- 思想简单，实现容易
- 许多强大的非线性模型的基础
- 结果具有很好的可解释性
- 蕴含机器学习中的很多重要思想

1

线性回归算法

监督学习分为回归和分类

✓ 回归 (Regression、Prediction)

标签连续

✓ 如何预测沈阳浑南的房价？

✓ 电影票房预估/股票市场预测？

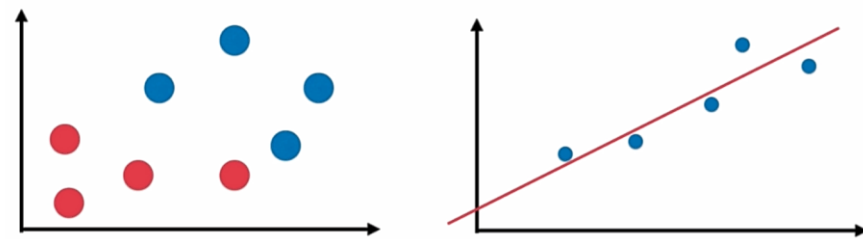
✓ 分类 (Classification)

标签离散

✓ 身高1.85m，体重100kg的男人穿什么尺码的T恤？

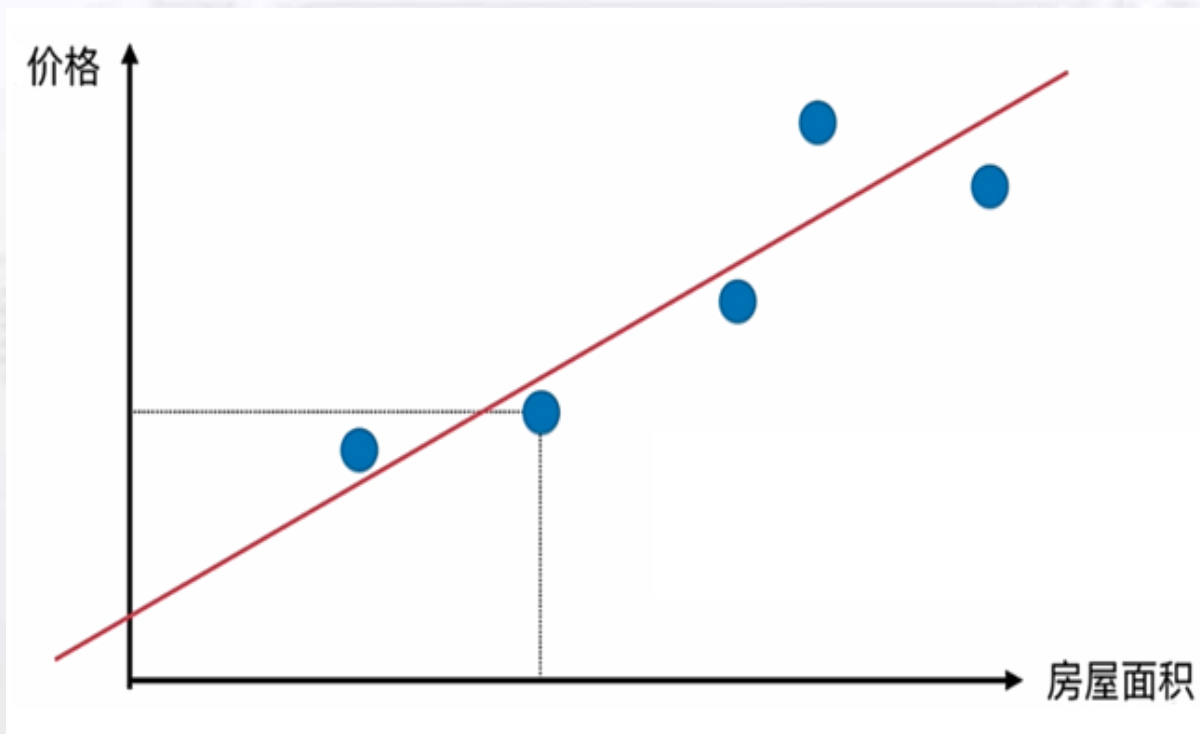
✓ 根据肿瘤的体积、患者的年龄来判断良性或恶性？

分类问题和回归问题



1

线性回归算法



线性回归模型是利用线性拟合的方式探寻数据背后的规律。即线性回归就是寻找一条直线，最大程度的“拟合” **样本特征**和**样本输出标记**之间的关系。

也可以说线性回归是一种通过属性的线性组合来进行预测的线性模型，其目的是找到**一条直线**或者**一个平面**或者**更高维的超平面**，使得预测值与真实值之间的误差最小化。

解决的是通过已知数据得到未知的结果（预测）。

样本特征只有一个，称为简单线性回归；
样本特征有多个，称为多元线性回归。

1

线性回归概念

m 代表训练集中样本的数量

n 代表特征的数量

x 代表特征/输入变量

y 代表目标变量/输出变量

(x, y) 代表训练集中的样本

$(x^{(i)}, y^{(i)})$ 代表第 i 个观察样本

h 代表学习算法的解决方案或函数也称为假设 (hypothesis)

$\hat{y} = h(x)$, 代表预测的值

建筑面积	总层数	楼层	实用面积	房价
143.7	31	10	105	36200
162.2	31	8	118	37000
199.5	10	10	170	42500
96.5	31	13	74	31200
.....				

$x^{(i)}$ 是特征矩阵中的第 i 行, 是一个向量。

上图的:

$$x^{(2)} = \begin{bmatrix} 162.2 \\ 31 \\ 8 \\ 118 \end{bmatrix} \quad y^{(2)} = 37000$$

$x_j^{(i)}$ 代表特征矩阵中第 i 行的第 j 个特征

上图的 $x_2^{(2)} = 31, x_3^{(2)} = 8$

1

线性回归概念

x 和 y 的关系

$$h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$



可以设 $x_0 = 1$

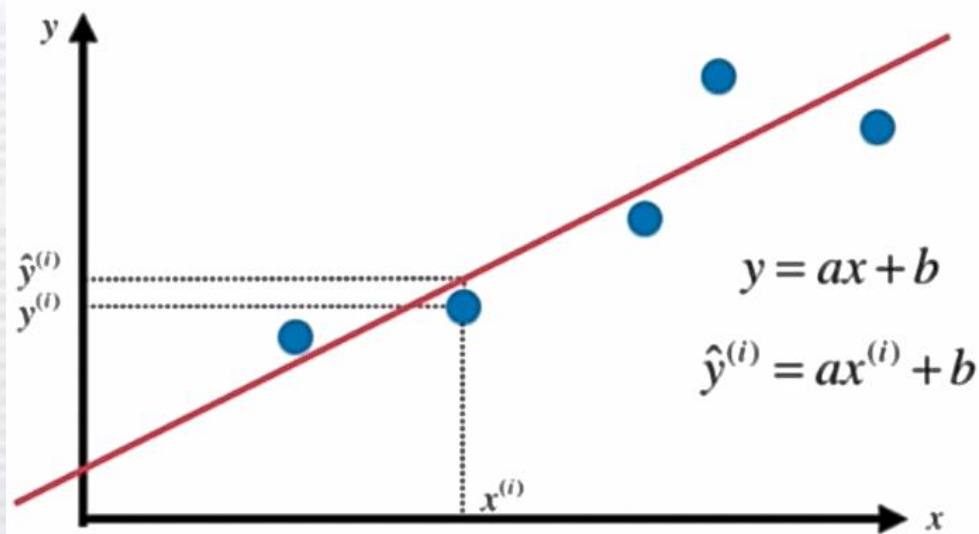
则：
$$h(x) = w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = w^T X$$

注意：若表达式 $h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ ，则 b 可以融入到 w_0

1

线性回归概念

示例：



假设我们找到了最佳拟合的直线方程：

$$y = ax + b$$

则对于每一个样本点 $x^{(i)}$

根据我们的直线方程，预测值为：

$$\hat{y}^{(i)} = ax^{(i)} + b$$

真值为： $y^{(i)}$

问题：如何表达预测结果的准确性？

1

线性回归算法

满足准确度的条件： $y^{(i)}$ 和 $\hat{y}^{(i)}$ 的差距尽量小

$$\begin{aligned} & y^{(i)} - \hat{y}^{(i)} \\ & | y^{(i)} - \hat{y}^{(i)} | \\ & (y^{(i)} - \hat{y}^{(i)})^2 \end{aligned}$$

考虑所有样本： $\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$

1

线性回归算法

目标：使 $\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$ 尽可能小

$$\hat{y}^{(i)} = ax^{(i)} + b$$

目标：找到a和b，使得 $\sum_{i=1}^m (y^{(i)} - ax^{(i)} - b)^2$ 尽可能小

$x^{(i)}$ 与 $y^{(i)}$ 是已知量，a和b是求解的未知量

由于是监督学习，多个样本会有多个 $x^{(i)}$ 与 $y^{(i)}$ ，相当于给出一组房产数据（面积—价格）

1

线性回归概念

$$h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

损失函数采用平方和损失：

$$l(x^{(i)}) = \frac{1}{2} (h(x^{(i)}) - y^{(i)})^2$$

要找到一组 $w(w_0, w_1, w_2, \dots, w_n)$,

$$\text{使得 } J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$$

(残差平方和) 最小

损失函数 (Loss Function) 度量单样本预测的错误程度，损失函数值越小，模型就越好。常用的损失函数包括：0-1损失函数、平方损失函数、绝对损失函数、对数损失函数等。

代价函数 (Cost Function) 度量全部样本集的平均误差。常用的代价函数包括均方误差、均方根误差、平均绝对误差等。

目标函数 (Object Function) 代价函数和正则化函数，最终要优化的函数。

1

线性回归算法

- 找到一组数据a和b，使目标函数值最小，这是典型的机器学习算法的推导思路；
- 换句话说，所谓的建模过程其实就是找到一个模型，最大程度地来拟合数据，在线性回归算法中，这个模型就是一个直线方程，另外，最大程度拟合数据，本质就是找到这样一个函数，即**损失函数**（用于度量所用模型没有拟合出样本的那一部分，即损失的部分），但是有的算法可能度量的是拟合的程度，即**效用函数**；
- 通过分析问题，确定问题的**目标函数：损失函数或者效用函数**；
- 通过**最优化**损失函数或者效用函数，获得机器学习的模型。
- 如前面讲的，机器学习算法有参数学习和非参数学习算法：几乎所有参数学习的算法，都是这种模式。所谓参数学习算法，指机器学习的任务就是学习这些模型的参数，而学习这些模型的参数，就是相应地找到这些参数，使得可以最优化损失函数或效用函数

目标：找到a和b，使得 $\sum_{i=1}^m (y^{(i)} - ax^{(i)} - b)^2$ 尽可能小



损失函数(loss function) 效用函数(utility function)

线性回归	SVM
多项式回归	神经网络
逻辑回归	

**区别：模型不同，目标函数不同，
优化方式不同**
最优化原理 / 凸优化

1

线性回归算法

优化方法一：最小二乘法 (Least Square Method)

目标：找到a和b，使得 $\sum_{i=1}^m (y^{(i)} - ax^{(i)} - b)^2$ 尽可能小

真实值 预测值

典型的最小二乘法问题：最小化误差的平方

$$a = \frac{\sum_{i=1}^m (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^m (x^{(i)} - \bar{x})^2}$$

x的均值 y的均值

$$b = \bar{y} - a\bar{x}$$

2

最小二乘法

目标：找到a和b，使得 $\sum_{i=1}^m (y^{(i)} - ax^{(i)} - b)^2$ 尽可能小


 $J(a,b)$

$J(a,b)$ ：损失函数

求函数的最小值，可对函数的未知变量求导，在导数为0的位置，即是最小值的位置

$$\frac{\partial J(a,b)}{\partial a} = 0 \quad \frac{\partial J(a,b)}{\partial b} = 0$$

2

最小二乘法

$$J(a,b) = \sum_{i=1}^m (y^{(i)} - ax^{(i)} - b)^2 \quad \frac{\partial J(a,b)}{\partial a} = 0 \quad \frac{\partial J(a,b)}{\partial b} = 0$$

$$\frac{\partial J(a,b)}{\partial b} = \sum_{i=1}^m 2(y^{(i)} - ax^{(i)} - b)(-1) = 0$$

$$\sum_{i=1}^m (y^{(i)} - ax^{(i)} - b) = 0$$

$$\sum_{i=1}^m y^{(i)} - a \sum_{i=1}^m x^{(i)} - \sum_{i=1}^m b = 0 \rightarrow \sum_{i=1}^m y^{(i)} - a \sum_{i=1}^m x^{(i)} - mb = 0$$

$$mb = \sum_{i=1}^m y^{(i)} - a \sum_{i=1}^m x^{(i)}$$

$$b = \bar{y} - a\bar{x}$$

$$\frac{\partial J(a,b)}{\partial a} = \sum_{i=1}^m 2(y^{(i)} - ax^{(i)} - b)(-x^{(i)}) = 0$$

$$\sum_{i=1}^m (y^{(i)} - ax^{(i)} - b)x^{(i)} = 0$$

$$\sum_{i=1}^m (y^{(i)} - ax^{(i)} - \bar{y} + a\bar{x})x^{(i)} = 0$$



$$\sum_{i=1}^m (x^{(i)}y^{(i)} - a(x^{(i)})^2 - x^{(i)}\bar{y} + a\bar{x}x^{(i)})$$

$$\sum_{i=1}^m (x^{(i)}y^{(i)} - x^{(i)}\bar{y} - a(x^{(i)})^2 + a\bar{x}x^{(i)})$$

$$\sum_{i=1}^m (x^{(i)}y^{(i)} - x^{(i)}\bar{y}) - \sum_{i=1}^m (a(x^{(i)})^2 - a\bar{x}x^{(i)})$$

$$\sum_{i=1}^m (x^{(i)}y^{(i)} - x^{(i)}\bar{y}) - a \sum_{i=1}^m ((x^{(i)})^2 - \bar{x}x^{(i)}) = 0$$

2

最小二乘法

$$a = \frac{\sum_{i=1}^m (x^{(i)}y^{(i)} - x^{(i)}\bar{y})}{\sum_{i=1}^m ((x^{(i)})^2 - \bar{x}x^{(i)})} \rightarrow \sum_{i=1}^m x^{(i)}\bar{y} = \bar{y} \sum_{i=1}^m x^{(i)} = m\bar{y} \cdot \bar{x} = \bar{x} \sum_{i=1}^m y^{(i)} = \sum_{i=1}^m \bar{x}y^{(i)} = \sum_{i=1}^m \bar{x} \cdot \bar{y}$$

$$= \frac{\sum_{i=1}^m (x^{(i)}y^{(i)} - x^{(i)}\bar{y} - \bar{x}y^{(i)} + \bar{x} \cdot \bar{y})}{\sum_{i=1}^m ((x^{(i)})^2 - \bar{x}x^{(i)} - \bar{x}x^{(i)} + \bar{x}^2)} = \frac{\sum_{i=1}^m (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^m (x^{(i)} - \bar{x})^2}$$

3

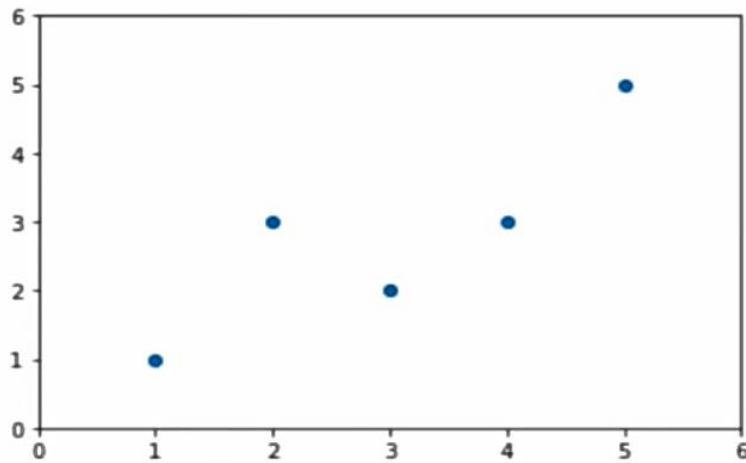
简单线性回归的实现

实现 Simple Linear Regression

```
In [1]: 1 import numpy as np
        2 import matplotlib.pyplot as plt
```

```
In [2]: 1 x = np.array([1., 2., 3., 4., 5.])
        2 y = np.array([1., 3., 2., 3., 5.])
```

```
In [3]: 1 plt.scatter(x, y)
        2 plt.axis([0, 6, 0, 6])
        3 plt.show()
```



$$a = \frac{\sum_{i=1}^m (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^m (x^{(i)} - \bar{x})^2} \quad b = \bar{y} - a\bar{x}$$

```
In [4]: 1 x_mean = np.mean(x)
        2 y_mean = np.mean(y)
```

```
In [5]: 1 num = 0.0
        2 d = 0.0
        3 for x_i, y_i in zip(x, y):
        4     num += (x_i - x_mean) * (y_i - y_mean)
        5     d += (x_i - x_mean) ** 2
```

```
In [6]: 1 a = num / d
        2 b = y_mean - a * x_mean
```

```
In [7]: 1 a
```

```
Out[7]: 0.80000000000000004
```

```
In [8]: 1 b
```

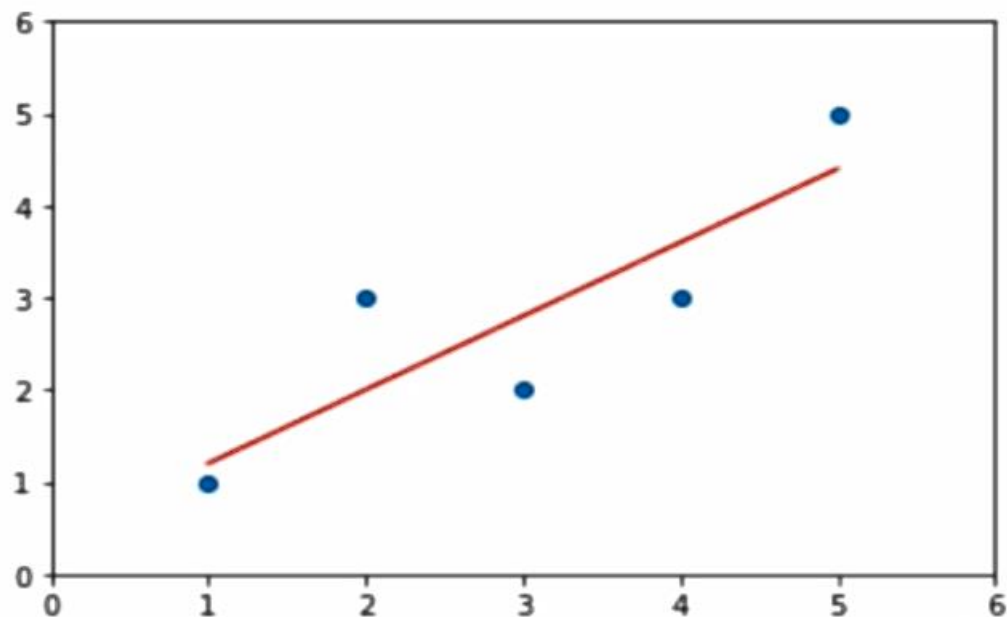
```
Out[8]: 0.39999999999999997
```

3

简单线性回归的实现

```
In [9]: 1 y_hat = a * x + b
```

```
In [10]: 1 plt.scatter(x, y)
2 plt.plot(x, y_hat, color='r')
3 plt.axis([0, 6, 0, 6])
4 plt.show()
```



```
In [11]: 1 x_predict = 6
2 y_predict = a * x_predict + b
```

```
In [12]: 1 y_predict
```

```
Out[12]: 5.2000000000000002
```

3

简单线性回归的实现

方法一:

```
1 import numpy as np
2
3
4 class SimpleLinearRegression1:
5
6     def __init__(self):
7         """初始化Simple Linear Regression 模型"""
8         self.a_ = None
9         self.b_ = None
10
11     def fit(self, x_train, y_train):
12         """根据训练数据集x_train,y_train训练Simple Linear Regression模型"""
13         assert x_train.ndim == 1, \
14             "Simple Linear Regressor can only solve single feature training data."
15         assert len(x_train) == len(y_train), \
16             "the size of x_train must be equal to the size of y_train"
17
18         x_mean = np.mean(x_train)
19         y_mean = np.mean(y_train)
20
21         num = 0.0
22         d = 0.0
23         for x, y in zip(x_train, y_train):
24             num += (x - x_mean) * (y - y_mean)
25             d += (x - x_mean) ** 2
26
27         self.a_ = num / d
28         self.b_ = y_mean - self.a_ * x_mean
29
30     return self
```

3

简单线性回归的实现

```
31
32 def predict(self, x_predict):
33     """给定待预测数据集x_predict, 返回表示x_predict的结果向量"""
34     assert x_predict.ndim == 1, \
35         "Simple Linear Regressor can only solve single feature training data."
36     assert self.a_ is not None and self.b_ is not None, \
37         "must fit before predict!"
38
39     return np.array([self._predict(x) for x in x_predict])
40
41 def _predict(self, x_single):
42     """给定单个待预测数据x_single, 返回x_single的预测结果值"""
43     return self.a_ * x_single + self.b_
44
45 def __repr__(self):
46     return "SimpleLinearRegression1()"
```

训练集意义：在fit()函数训练模型的参数

3

简单线性回归的实现

```
In [14]: 1 from playML.SimpleLinearRegression import SimpleLinearRegression1
          2
          3 reg1 = SimpleLinearRegression1()
          4 reg1.fit(x, y)
```

```
Out[14]: SimpleLinearRegression1()
```

```
In [15]: 1 reg1.predict(np.array([x_predict]))
```

```
Out[15]: array([ 5.2])
```

```
In [16]: 1 reg1.a_
```

```
Out[16]: 0.80000000000000000004
```

```
In [17]: 1 reg1.b_
```

```
Out[17]: 0.399999999999999947
```

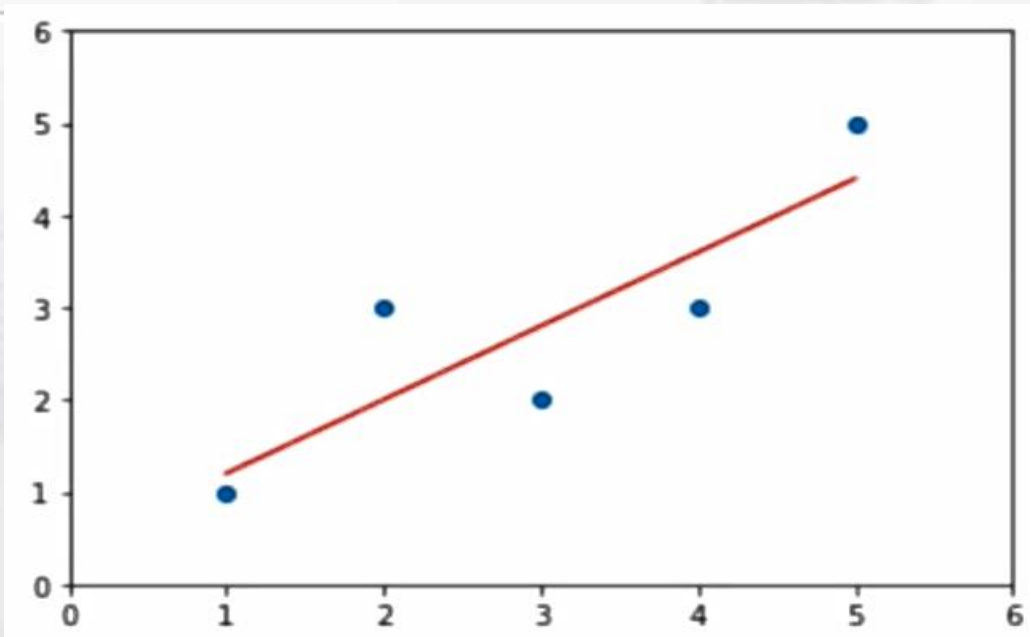

3

简单线性回归的实现

```
In [18]: 1 y_hat1 = reg1.predict(x)
```

```
In [19]: 1 plt.scatter(x, y)
2 plt.plot(x, y_hat1, color="r")
3 plt.axis([0, 6, 0, 6])
4 plt.show()
```

此方法性能较低



4

向量化

方法二：向量化运算

由于使用for循环进行计算，程序运行性能效率较低而使用向量运算，性能会得到很大提升

$$a = \frac{\sum_{i=1}^m (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^m (x^{(i)} - \bar{x})^2}$$

$$\sum_{i=1}^m w^{(i)} \cdot v^{(i)}$$



$w \cdot v$ 向量的点乘

$$w = (w^{(1)}, w^{(2)}, \dots, w^{(m)})$$

$$v = (v^{(1)}, v^{(2)}, \dots, v^{(m)})$$

4

向量化

方法二：向量化运算

由于使用for循环进行计算，程序运行性能效率较低
而使用向量运算，性能会得到很大提升

```
def fit(self, x_train, y_train):  
    """根据训练数据集x_train,y_train训练Simple Linear Regression模型"""  
    assert x_train.ndim == 1, \  
        "Simple Linear Regressor can only solve single feature training data."  
    assert len(x_train) == len(y_train), \  
        "the size of x_train must be equal to the size of y_train"  
  
    x_mean = np.mean(x_train)  
    y_mean = np.mean(y_train)  
  
    num = 0.0  
    d = 0.0  
    for x, y in zip(x_train, y_train):  
        num += (x - x_mean) * (y - y_mean)  
        d += (x - x_mean) ** 2  
  
    self.a_ = num / d  
    self.b_ = y_mean - self.a_ * x_mean  
  
    return self
```

```
num = (x_train - x_mean).dot(y_train - y_mean)  
d = (x_train - x_mean).dot(x_train - x_mean)
```


4

向量化

向量化实现SimpleLinearRegression

```
In [19]: 1 from playML.SimpleLinearRegression import SimpleLinearRegression2
```

```
In [20]: 1 reg2 = SimpleLinearRegression2()  
2 reg2.fit(x, y)
```

```
Out[20]: SimpleLinearRegression2()
```

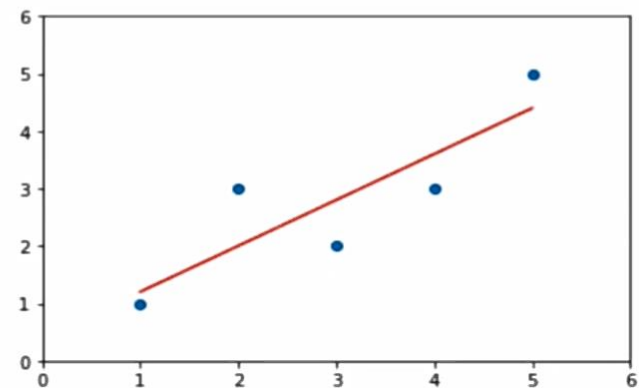
```
In [21]: 1 reg2.a_
```

```
Out[21]: 0.80000000000000004
```

```
In [22]: 1 reg2.b_
```

```
Out[22]: 0.39999999999999947
```

```
In [23]: 1 y_hat2 = reg2.predict(x)  
2  
3 plt.scatter(x, y)  
4 plt.plot(x, y_hat2, color='r')  
5 plt.axis([0, 6, 0, 6])  
6 plt.show()
```



4

向量化

向量化实现的性能测试

```
In [24]: 1 m = 1000000  
2 big_x = np.random.random(size=m)  
3 big_y = big_x * 2.0 + 3.0 + np.random.normal(size=m)
```

```
In [25]: 1 %timeit reg1.fit(big_x, big_y)  
2 %timeit reg2.fit(big_x, big_y)  
  
1 loop, best of 3: 1.1 s per loop  
10 loops, best of 3: 20 ms per loop
```

```
In [26]: 1 reg1.a_
```

```
Out[26]: 2.0028631069011333
```

```
In [27]: 1 reg1.b_
```

```
Out[27]: 2.9990570839126409
```

```
In [28]: 1 reg2.a_
```

```
Out[28]: 2.0028631069010903
```

```
In [29]: 1 reg2.b_
```

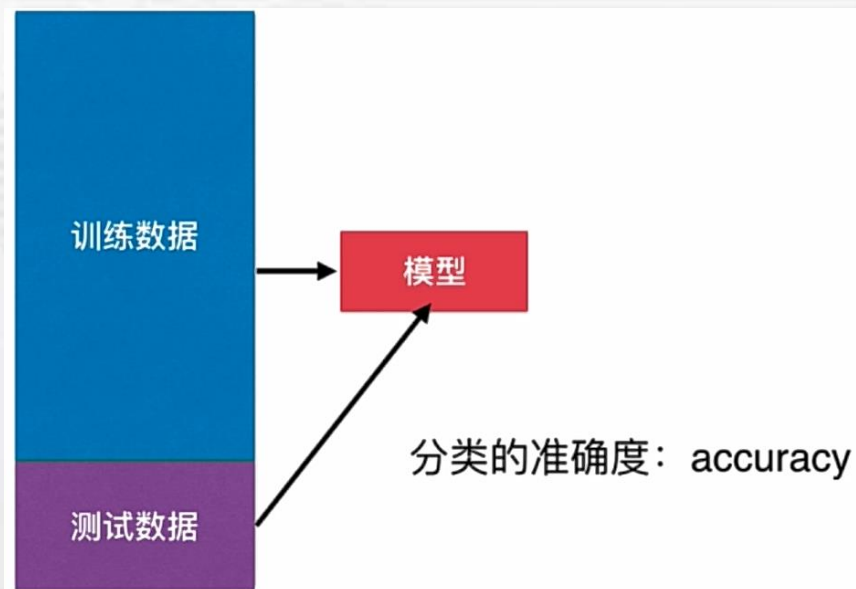
```
Out[29]: 2.9990570839126627
```

5

衡量线性回归法的指标

模型搭建完后，还需要对模型进行评估：

评价一个算法所使用的标准和训练模型时最优化的目标函数，是可以完全不一致的



目标：找到a和b，使得 $\sum_{i=1}^m (y^{(i)} - ax^{(i)} - b)^2$ 尽可能小

$$\sum_{i=1}^m (y_{train}^{(i)} - ax_{train}^{(i)} - b)^2$$

$$\sum_{i=1}^m (y_{train}^{(i)} - \hat{y}_{train}^{(i)})^2$$

$$\hat{y}_{test}^{(i)} = ax_{test}^{(i)} + b$$

衡量标准： $\sum_{i=1}^m (y_{test}^{(i)} - \hat{y}_{test}^{(i)})^2$

5

· 衡量线性回归法的指标

主要有两种评判标准：R-squared和Adj.R-squared。

以上评判标准主要用来衡量线性拟合的优劣，遵循两条规则：

- ✓ R-squared和Adj.R-squared的取值范围是0~1；
- ✓ 它们的值越接近1，表明模型的拟合程度越高。

下面具体讲解这两种评判标准。

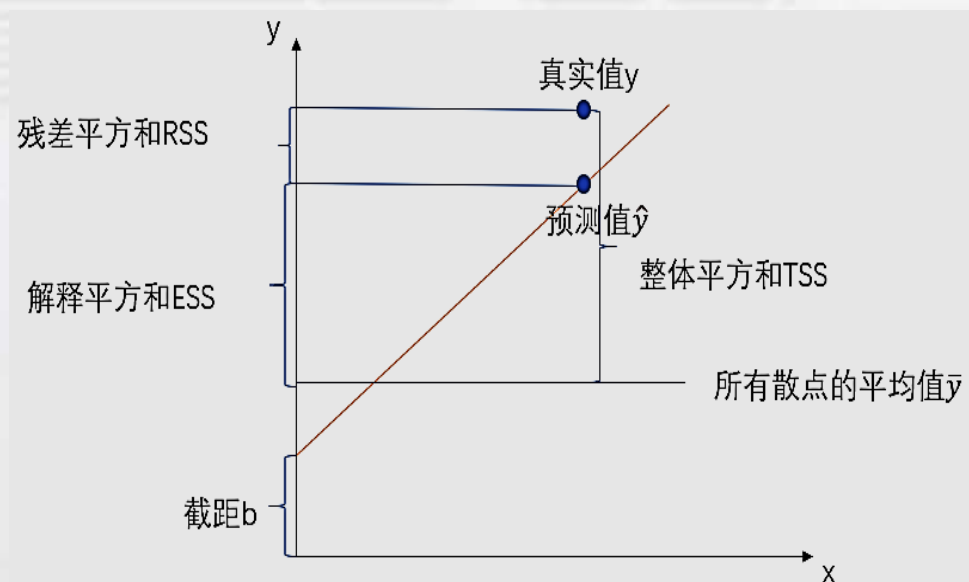
5

衡量线性回归法的指标

(1) R-squared

要想理解R-squared，首先得了解3组新的概念：

整体平方和TSS，残差平方和RSS和解释平方和ESS。他们三者的关系如下图所示（为了使图看得更简洁这里没有绘制样本点）。



由上图可知： $RSS + ESS = TSS$ ，
即：残差平方和 + 解释平方和 = 整体平方和。

整体平方和： $TSS = \sum (y - \bar{y})^2$

残差平方和： $RSS = \sum (y - \hat{y})^2$

解释平方和： $ESS = \sum (\hat{y} - \bar{y})^2$

$$R^2 = 1 - \frac{RSS}{TSS}$$

$\frac{RSS}{TSS}$ 用来表示训练出来的模型产生错误的概率。

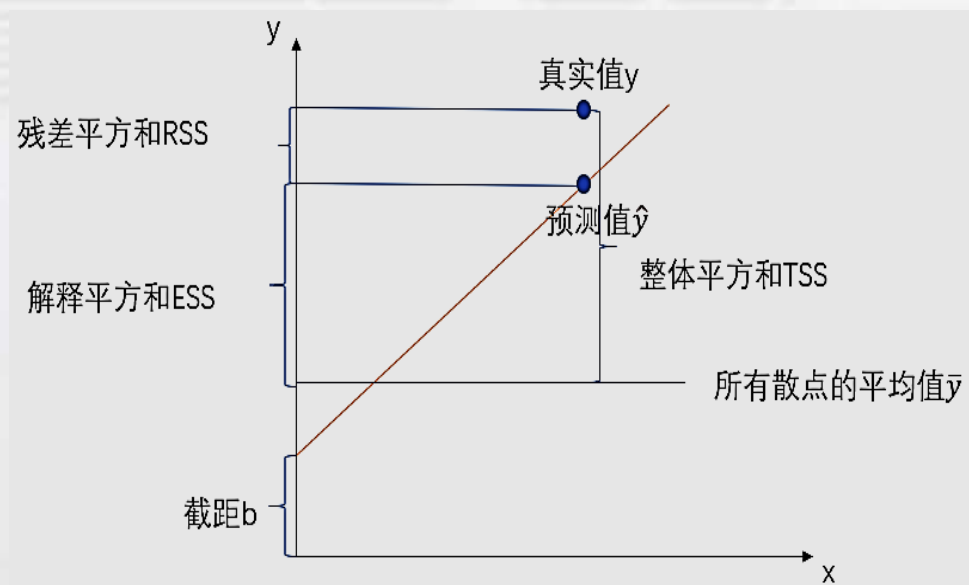
5

衡量线性回归法的指标

(1) R-squared

$$R^2 = 1 - \frac{RSS}{TSS},$$

$\frac{RSS}{TSS}$ 用来表示训练出来的模型产生错误的概率。



对于一个拟合程度较高的线性回归模型，希望其实际值和预测值越接近越好，这表明训练的模型的拟合程度较高。即用来衡量拟合数据程度的标准——残差平方和RSS尽可能小。当RSS趋近于0时，说明实际值基本上都落在了拟合曲线上，此时模型的拟合程度非常高， R^2 趋近于1。当预测模型不犯任何错误时， $R^2=1$ （这里需要注意：拟合程度不是越高越好，拟合程度过高可能会导致过拟合现象）。当 $R^2<0$ 时，说明 $RSS>TSS$ ，此时我们的数据很有可能不存在任何线性关系。

5

衡量线性回归法的指标

(2) Adj.R-squared

R-squared 正常情况下取值范围为0-1，描述的是输入变量对输出变量的解释程度，在单变量线性回归中R-square越大，越接近1越好。然而只要增加了更多的变量，无论增加的变量是否和输出存在关系，则R-squared要么保持不变，要么增加。所以需要adjusted R-squared，它会对那些增加的且不会改善模型效果的变量增加一个惩罚项。其公式如下。

$$R^{2'} = 1 - \frac{(1 - R^2)(n - 1)}{n - k - 1}$$

其中n为样本数量，k为特征变量数量，从上述公式可以看出，特征变量数量k越大，其实会对Adj.R-squared产生负影响，这说明不要为了追求高R-squared值而添加过多的特征变量。在考虑了特征变量数量之后，Adj.R-squared就能够更准确地反映线性模型的拟合程度。由此得到以下结论：**如果单变量线性回归，则使用 R-squared评估，多变量线性回归则使用Adj. R-squared对模型进行评估。**

5

· 衡量线性回归法的指标

另外还有以下三种指标都可以用来评估线性回归模型。

线性回归算法的评测

- 均方误差 MSE : $\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$ (Mean Square Error, MSE)
- 均方根误差 RMSE : $\sqrt{\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2} = \sqrt{MSE}$ (Root Mean Square Error, RMSE)
- 平均绝对误差 MAE : $\frac{1}{m} \sum_{j=1}^m |y^{(\tilde{i})} - \hat{y}^{(\tilde{i})}|$ (Mean Absolute Error, MAE)

其中， $y^{(i)}$ 和 $\hat{y}^{(i)}$ 分别表示第 i 个样本的真实值和预测值， m 为样本个数。

5

衡量线性回归法的指标

实例演示

05 衡量回归算法的标准

```
In [1]: 1 import numpy as np
        2 import matplotlib.pyplot as plt
        3 from sklearn import datasets
```

波士顿房产数据

```
In [2]: 1 boston = datasets.load_boston()
```

```
In [3]: 1 print(boston.DESCR)
```

Boston House Prices dataset

=====

Notes

Data Set Characteristics:

:Number of Instances: 506

:Number of Attributes: 13 numeric/categorical predictive

:Median Value (attribute 14) is usually the target

:Attribute Information (in order):

- CRIM	per capita crime rate by town
- ZN	proportion of residential land zoned for lots over 25,000 sq.ft.
- INDUS	proportion of non-retail business acres per town
- CHAS	Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
- NOX	nitric oxides concentration (parts per 10 million)
- RM	average number of rooms per dwelling

5

衡量线性回归法的指标

实例演示

```
In [4]: 1 boston.feature_names
```

```
Out[4]: array(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD',  
              'TAX', 'PTRATIO', 'B', 'LSTAT'],  
             dtype='<U7')
```

```
In [5]: 1 x = boston.data[:,5] # 只使用房间数量这个特征
```

```
In [6]: 1 x.shape
```

```
Out[6]: (506,)
```

```
In [7]: 1 y= boston.target
```

```
In [8]: 1 y.shape
```

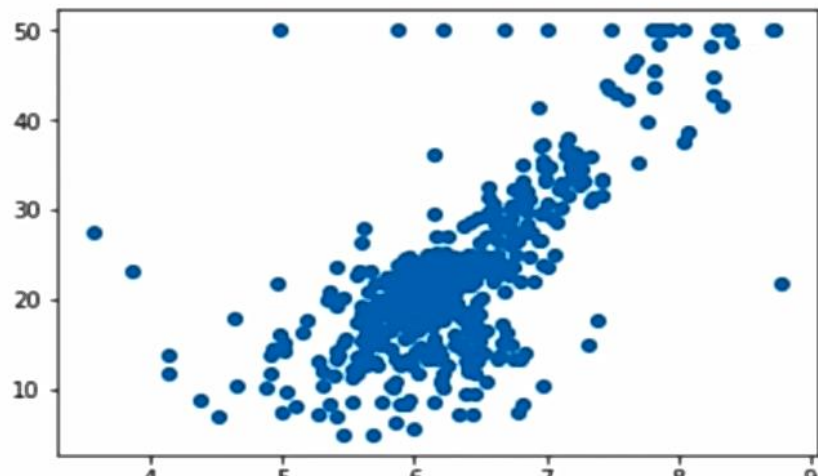
```
Out[8]: (506,)
```

5

衡量线性回归法的指标

实例演示

```
In [9]: 1 plt.scatter(x, y)
        2 plt.show()
```

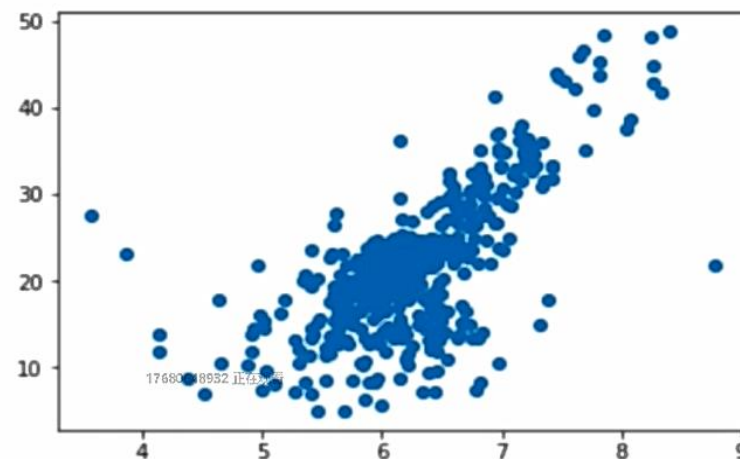


```
In [10]: 1 np.max(y)
```

Out[10]: 50.0

```
In [11]: 1 x = x[y < 50.0]
        2 y = y[y < 50.0]
```

```
In [12]: 1 plt.scatter(x, y)
        2 plt.show()
```



5

衡量线性回归法的指标

实例演示

使用简单线性回归法

```
In [13]: 1 from playML.model_selection import train_test_split  
2  
3 x_train, x_test, y_train, y_test = train_test_split(x, y, seed=666)
```

```
In [14]: 1 x_train.shape
```

```
Out[14]: (392,)
```

```
In [15]: 1 x_test.shape
```

```
Out[15]: (98,)
```

```
In [17]: 1 from playML.SimpleLinearRegression import SimpleLinearRegression
```

```
In [18]: 1 reg = SimpleLinearRegression()  
2 reg.fit(x_train, y_train)
```

```
Out[18]: SimpleLinearRegression()
```

```
In [19]: 1 reg.a_
```

```
Out[19]: 7.8608543562689555
```

```
In [20]: 1 reg.b_
```

```
Out[20]: -27.459342806705543
```

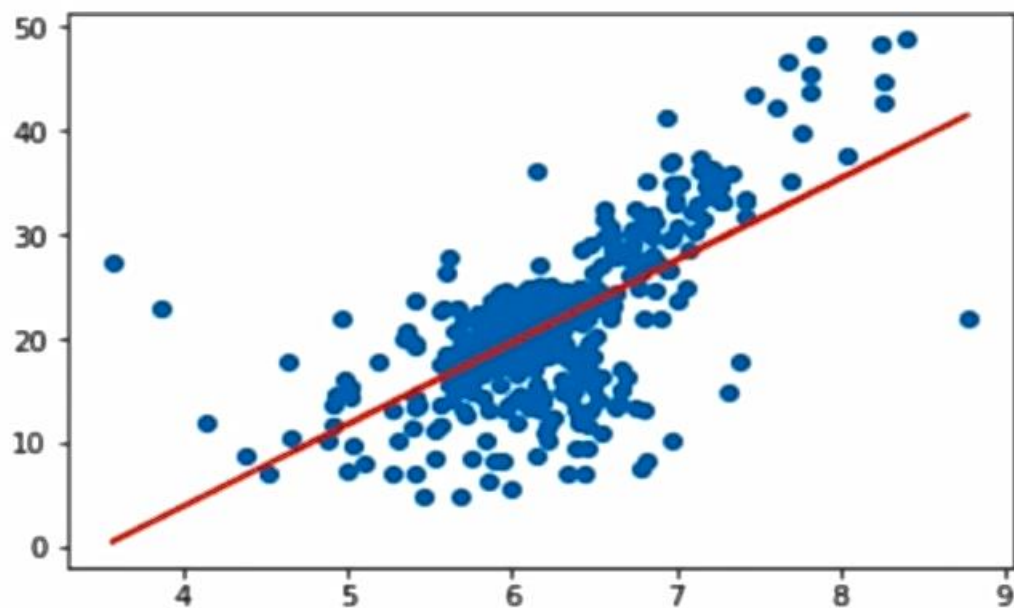
5

衡量线性回归法的指标

实例演示

In [21]:

```
1 plt.scatter(x_train, y_train)
2 plt.plot(x_train, reg.predict(x_train), color="r")
3 plt.show()
```



In [22]:

```
1 y_predict = reg.predict(x_test)
```


5

衡量线性回归法的指标

实例演示

MSE

```
In [23]: 1 mse_test = np.sum((y_predict - y_test)**2) / len(y_test)
          2 mse_test
```

```
Out[23]: 24.156602134387438
```

RMSE

```
In [25]: 1 from math import sqrt
          2
          3 rmse_test = sqrt(mse_test)
          4 rmse_test
```

```
Out[25]: 4.914936635846635
```

MAE

```
In [26]: 1 mae_test = np.sum(np.absolute(y_predict - y_test)) / len(y_test)
          2 mae_test
```

```
Out[26]: 3.5430974409463873
```


5

• 衡量线性回归法的指标

RMSE vs MAE

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_{test}^{(i)} - \hat{y}_{test}^{(i)})^2}$$

$$MAE = \frac{1}{m} \sum_{i=1}^m |y_{test}^{(i)} - \hat{y}_{test}^{(i)}|$$

均方误差 (Mean Square Error, MSE)

均方根误差 RMSE(Root Mean Square Error, RMSE)

平均绝对误差 (Mean Absolute Error, MAE)

MSE vs MAE

实际应用中，选择 MSE 还是MAE 呢？从计算机求解梯度的复杂度来说，MSE要优于MAE，而且梯度也是动态变化的，能较快准确地达到收敛。但是从离群点角度来看，如果离群点是实际数据或重要数据，而且是应该被检测到的异常值，那么应该使用MSE。另一方面，离群点仅仅代表数据损坏或者错误采样，无须给予过多关注，那么应该选择MAE作为损失。



谢谢